



BASES DE DONNÉES

CM

Langage de Définition de Données (LDD)

Vincent COUTURIER

Maitre de conférences – Université Savoie Mont-Blanc

vincent.couturier@univ-smb.fr

Préambule

- Schéma de la base de données EMPLOYES :

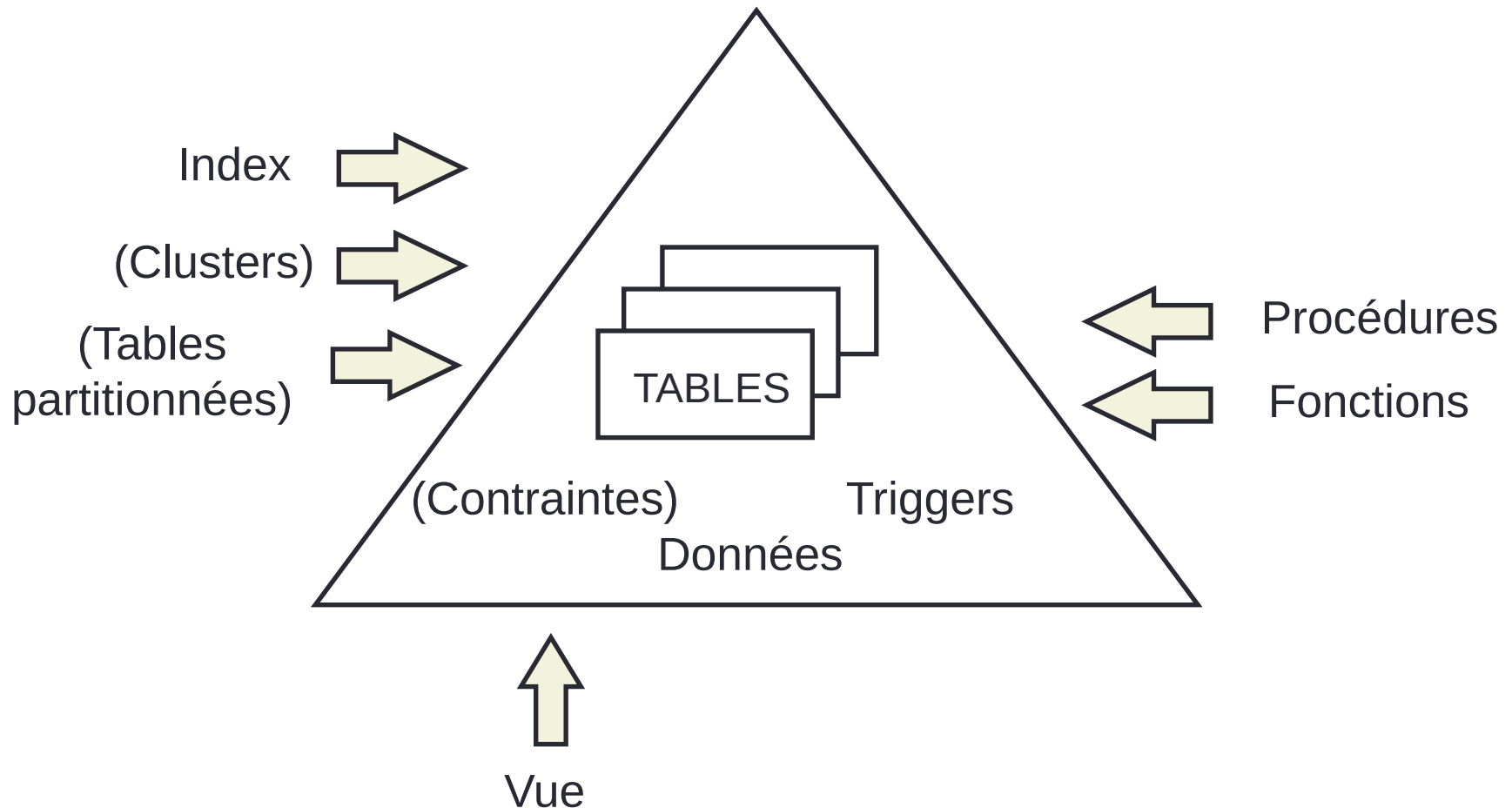
DEPT(DEPTNO, DNAME, LOC)

EMP(EMPNO, ENAME, JOB, #MGR, HIREDATE,
SAL, COMM, ADR, #DEPTNO)

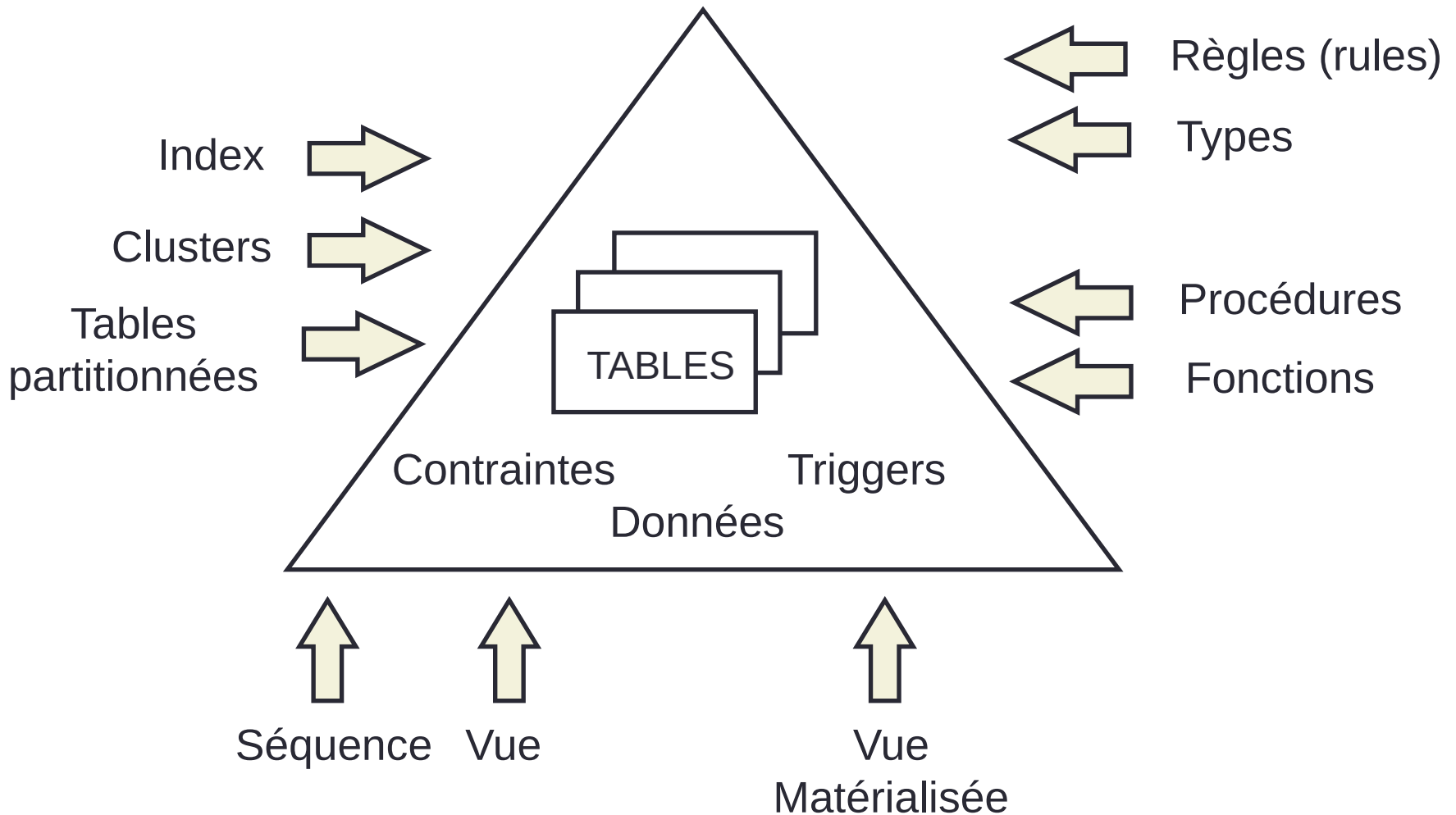
Objectifs LDD

- Objectifs :
 - CREATE (création) :
 - Tables avec des colonnes de types différents et contraintes (CREATE TABLE)
 - Séquence (CREATE SEQUENCE)
 - Vue (CREATE VIEW)
 - Index (CREATE INDEX)
 - ALTER (modification) :
 - Tables et contraintes (ALTER TABLE)
 - Séquence (ALTER SEQUENCE)
 - Vue (ALTER VIEW)
 - Index (ALTER INDEX)
 - DROP (suppression) :
 - Tables et contraintes (DROP TABLE)
 - Séquence (DROP SEQUENCE)
 - Vue (DROP VIEW)
 - Index (DROP INDEX)

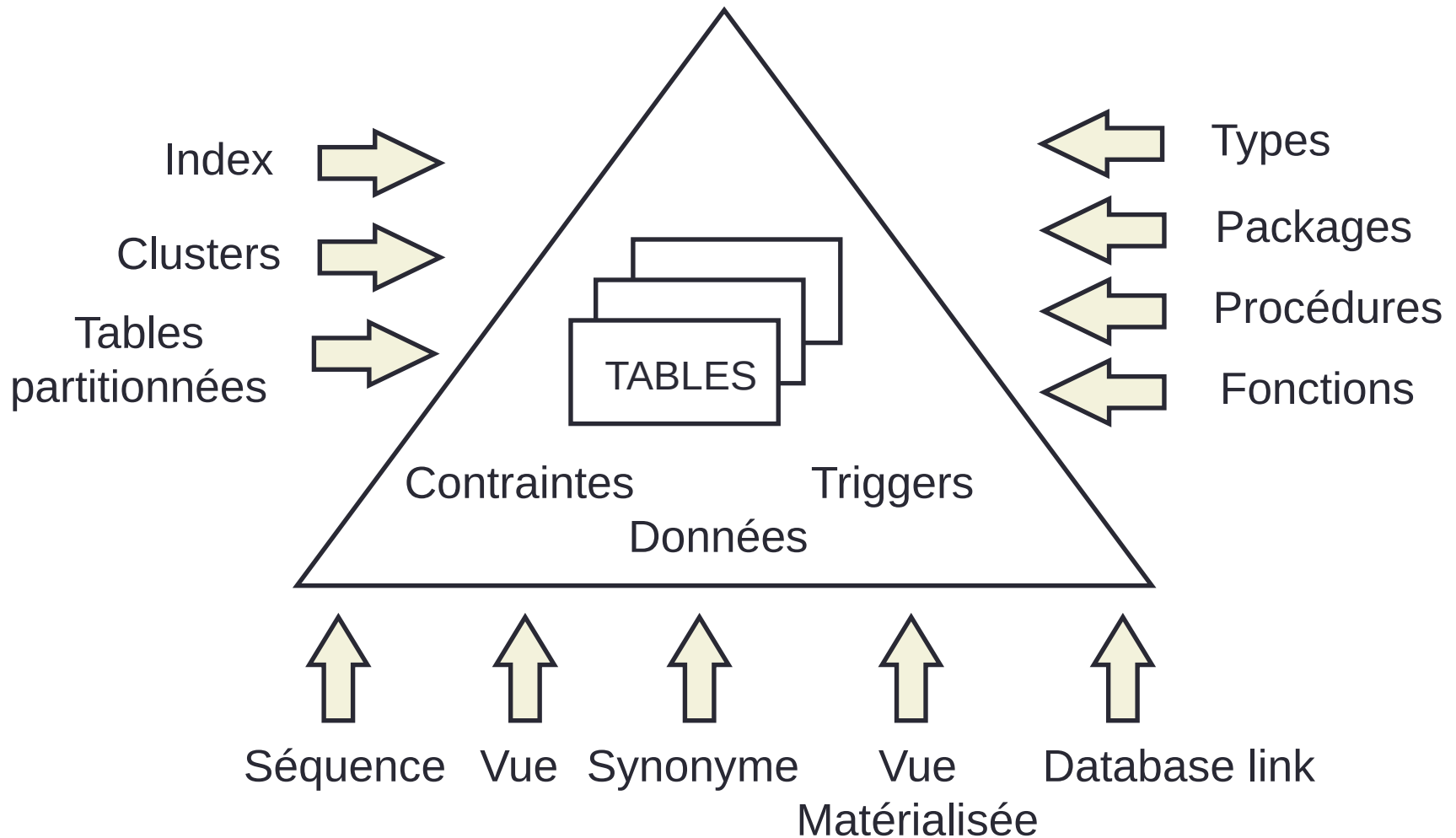
MySQL : Objets gérés



PostgreSQL : Objets gérés



Oracle : Objets gérés



TABLE

Création de tables

- Syntaxe :

```
CREATE TABLE nom_table (  

    nom_champ1 type_champ1,  

    nom_champ2 type_champ2,  

    ...  

    nom_champn type_champn) ;
```
- Exemple : création de la table des services

```
CREATE TABLE DEPT  

(  

    DEPTNO          NUMERIC(2) NOT NULL,  

    DNAME           VARCHAR(14),  

    LOC             VARCHAR(13) DEFAULT 'PARIS'  

);
```

Nom de colonne	Type de donnée	Propriété (défaut = NULL)	Valeur par défaut (facultatif)
DEPTNO DNAME LOC	NUMERIC(2) VARCHAR(14) VARCHAR(13)	NOT NULL	DEFAULT 'PARIS'

Création de tables : choix des champs

- Avant de créer une table, il faut choisir les champs que l'on va utiliser. Pour chaque champ il faut choisir un *nom* et un *type*.
- Les champs peuvent être des types suivants : *numérique* (entier ou rationnel), *date et heure*, *chaîne de caractères*, *texte*, *blob*, *énuméré*, *ensemble*.
- Types possibles :
 - MySQL : CHAR(M), VARCHAR(M), INT[(M)] ou INTEGER[(M)], FLOAT[(M,D)], DOUBLE[(M,D)] ou REAL [(M,D)], DECIMAL[(M[,D])], ou NUMERIC[(M[,D])], DATE, TIME, DATETIME, TIMESTAMP[(M)], BLOB, ENUM, SET, ...
 - Oracle : CHAR(M), VARCHAR2(M), NUMBER[(M[,D])], DATE, TIMESTAMP[(M)], BLOB, CLOB, BFILE, ...
 - SQL SERVER : BIT, CHAR(M), VARCHAR(M), TEXT, DECIMAL[(M[,D])], FLOAT, INT, MONEY, REAL, DATETIME, SMALLDATETIME, TIMESTAMP[(M)], IMAGE, BINARY(M), UNIQUEIDENTIFIER, ...

Principaux types de données PostgreSQL (1)

- Types caractères:
 - CHAR(n) : Chaîne de caractères de longueur fixe : n octets au maximum (chaîne toujours complétée à droite par des espaces pour arriver à n octets).
 - VARCHAR(n) : Chaîne de caractères de longueur variable : n octets au maximum
 - TEXT : Chaîne de caractères de longueur illimitée ($\Leftrightarrow$ VARCHAR)
- Types numériques :
 - NUMERIC(p, s) : Numérique avec une précision de p chiffres dont s décimales
 - SMALLINT (de -32768 à +32767), INTEGER (de -214 7483 648 à +214 7483 647), REAL,...

Principaux types de données PostgreSQL (2)

- Types date :
 - DATE : Date comprise entre 1er janvier 4713 avant JC et le 31 décembre 5874897 après JC.
 - TIMESTAMP(p) : Données de type Date (Année, mois, jour, heure, minute et seconde) dans laquelle il est possible de préciser, à l'aide de la précision p, le nombre de chiffres significatifs pour les fractions de secondes (0 à 6).
- BOOLEAN
- BYTEA ($\Leftrightarrow$ BLOB) : Données binaires non structurées (4 Go maximum).
- ...

Autres types

- PostgreSQL intègre des fonctionnalités NoSQL depuis la version 9.3 :
 - Version 9.3 : Type de données JSON ⇔ Clé/valeur
 - 1 clé -> 1 document JSON
 - Version 9.4 : Type de données JSONB ⇔ orienté-documents
 - 1 clé -> 1 Document JSON (stocké au format binaire)
 - Chaque champ du JSON est adressable (« requêtable ») par une clé.
- Il est ainsi possible :
 - D'ajouter un (ou plusieurs) champ supplémentaire dans une table qui contiendra un document JSON.
 - Permet de mixer relationnel et NoSQL

Création de tables (suite)

- CREATE à l'aide de SELECT
 - Exemple : création de la table dept_20. La structure de cette table est identique à celle de la table DEPT

```
CREATE TABLE dept_20 AS  
  SELECT * FROM dept  
  WHERE deptno = 20
```

Création de contraintes d'intégrité (1)

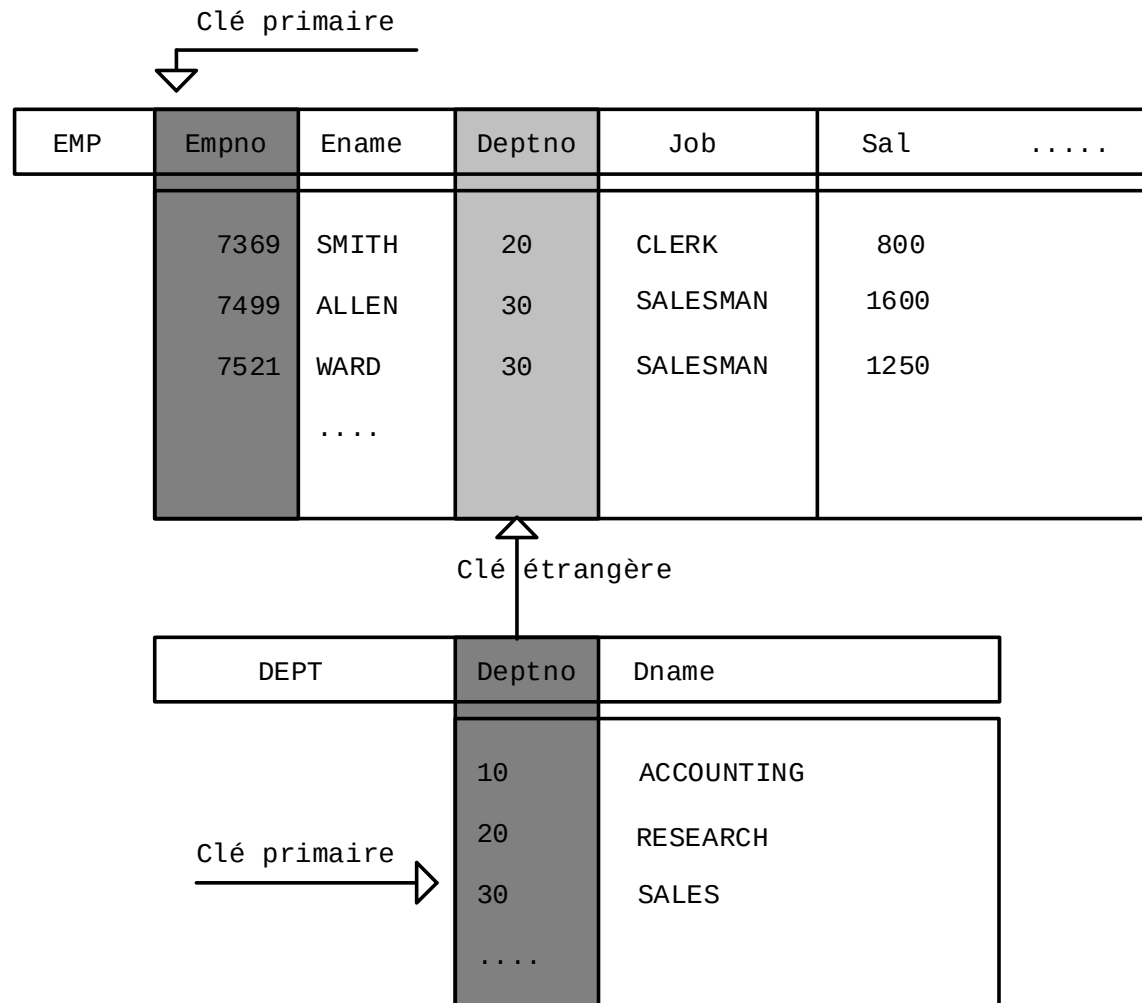
- Moyen permettant de garantir que les modifications apportées à la base ne pourront en aucun cas la rendre incohérente.
- Différents types de contraintes d'intégrité :
 - Définie lors de la création des tables
 - les contraintes de domaine
 - type de colonne, valeur par défaut, ensemble de valeurs, conditions qu'une valeur doit remplir ; caractère obligatoire ou non des colonnes (NULL ou NOT NULL).
 - les contraintes de clé unique
 - garantissent qu'une même valeur ne peut se trouver sur plus d'une ligne.
 - les contraintes de clé primaire
 - colonne ou groupe de colonnes choisis pour identifier de façon unique chacune des occurrences de tables, et référençable par une clé étrangère.
 - les contraintes référentielles (clé étrangère)
 - garantissent qu'une colonne ou un groupe de colonnes existe dans une autre entité.
 - Définies par trigger
 - Contraintes **temporelles** : salaire ne peut pas baisser...
 - Contraintes avec **agrégats** : ne porte pas sur un attribut ou un tuples, mais plusieurs tuples ou même toute la table :
 - Exemple : « Il doit y avoir autant de départements localisés à Paris que de départements à Londres »
 - Cette contrainte ne peut être vérifiée que lorsque « tous les départements sont insérés ».

Création de contraintes d'intégrité (2)

- Respect des contraintes d'intégrité :
 - Lors de chaque accès en mise à jour (ajout, modification, suppression), le SGBD doit **vérifier** les contraintes d'intégrité.
 - Elles sont en général définies lors de la **création** des tables, en donnant des précisions sur les attributs concernés.
 - Certaines font l'objet de procédures particulières appelées **triggers** (déclencheur) qui sont exécutées lors de l'accès en MAJ aux données.

Création de contraintes :

Clé primaire et clé étrangère



Création de contraintes :

Clé primaire et clé étrangère (suite)

- Lors d'une insertion :
 - Unicité de la clé primaire et des clés uniques ;
 - Pour chacune des clés étrangères, existence d'une occurrence correspondante dans la table référencée ;
 - Cohérence des valeurs avec leur type (NUMERIC, VARCHAR,...), le caractère obligatoire ou non (NULL ou NOT NULL), les conditions éventuelles.
- Lors d'une suppression :
 - S'il n'existe pas de clé étrangère dans d'autres tables dont la valeur correspond à l'une des clés primaires supprimées, la suppression est effectuée ;
 - Sinon,
 - soit la suppression est rejetée (ON DELETE RESTRICT)
 - Soit toutes les lignes référençant l'une des clés primaires supprimées sont aussi supprimées (suppression en CASCADE => ON DELETE CASCADE).
 - Soit les clés étrangères sont mises à NULL (ON DELETE SET NULL)

Création de contraintes :

Clé primaire et clé étrangère (suite)

- Lors d'une mise à jour :
 - Cohérence des valeurs avec leur type (NUMERIC, VARCHAR,...), le caractère obligatoire ou non (NULL ou NOT NULL), les conditions éventuelles.
 - Pour les mises à jour de clés étrangères, on devra réaliser le même contrôle que lors d'une insertion ;
 - Pour les mises à jour d'une clé primaire,
 - S'il n'existe pas de clé étrangère dans d'autres tables dont la valeur correspond à l'une des clés primaires mises à jour, la mise à jour est effectuée ;
 - Sinon :
 - Soit la mise à jour est rejetée (ON UPDATE RESTRICT : interdiction si clé utilisée)
 - Soit la mise à jour est répercutée sur toutes les lignes référençant la clé primaire modifiée (ON UPDATE CASCADE).
 - Soit les valeurs de la clé étrangère sont remplacées par NULL (ON UPDATE SET NULL)

Création de contraintes :

Contraintes de domaine

- Les types de données :
 - CHAR, VARCHAR, NUMERIC, ...
- Le caractère obligatoire ou facultatif d'une colonne :
 - NULL ou NOT NULL
- La clause DEFAULT :
 - valeur par défaut pour une colonne, cette valeur est utilisée en création (insert) ou modification (update) si l'utilisateur ne renseigne pas cette colonne,
 - peut être une fonction (Ex. : CURRENT_DATE).
- La clause CHECK (ou contraintes de validation) :
 - Contrôle de valeur effectué pour toute exécution d'une commande update, insert sur chaque ligne de la table
 - Si contrôle est négatif, ordre SQL annulé
 - Contrainte sur colonne :
 - `CONSTRAINT Ck_LIG_CDE check (qte_cdee > 0)`
 - Contrainte sur table (plusieurs colonnes impliquées) :
 - `CONSTRAINT CK_LIG_CDE check (qte_cdee >= qte_livree)`

Création de contraintes :

Contraintes de domaine (suite)

- Exemple : contraintes déclaratives

```
CREATE TABLE emp1
(
  empno      NUMERIC(4) NOT NULL,
  ename      VARCHAR(10),
  job        VARCHAR(9) CHECK (job in ('SALESMAN','CLERK','MANAGER')),
  mgr        NUMERIC(4),
  hiredate   DATE DEFAULT CURRENT_DATE,
  sal        NUMERIC(7,2) CHECK (sal > 300 AND sal < 9000),
  comm       NUMERIC(7,2) CHECK (comm IS NULL OR comm<=sal/2),
  deptno     NUMERIC(2) NOT NULL
);
```

- Remarque : il est conseillé de nommer les contraintes afin de simplifier le décodage des messages d'erreurs, la gestion des activations ou désactivations des contraintes. Dans ce cas, il est d'usage de les positionner à la fin du CREATE TABLE.

Création de contraintes :

Contraintes de domaine (suite)

- Exemple : contraintes déclaratives (MIEUX ECRIT !)

```
CREATE TABLE emp1
(
  empno      NUMERIC(4) NOT NULL,
  ename      VARCHAR(10),
  job        VARCHAR(9),
  mgr        NUMERIC(4),
  hiredate   DATE DEFAULT CURRENT_DATE,
  sal        NUMERIC(7,2),
  comm       NUMERIC(7,2),
  deptno     NUMERIC(2) NOT NULL,
  CONSTRAINT ck_emp1_job CHECK (job in
    ('SALESMAN', 'CLERK', 'MANAGER')),
  CONSTRAINT ck_emp1_sal CHECK (sal > 300 AND sal < 9000),
  CONSTRAINT ck_emp1_comm CHECK (comm IS NULL OR comm<=sal/2)
);
```

Création de contraintes :

Contraintes de domaine (suite)

- Exemple : Insertion d'une ligne ne respectant pas la contrainte sur la colonne JOB.

La contrainte portant sur la colonne job n'est pas nommée, le SGBD génère automatiquement un nom unique.

```
INSERT INTO emp1  
VALUES (7600, 'TOTO', 'INCONNU', 10, null, 0, 0, 10)  
ERROR at line 1:  
check constraint (SYS_C0011195) violated
```

- Exemple : Insertion d'une ligne ne respectant pas la contrainte sur la colonne salaire (contrainte nommée)

```
INSERT INTO emp1 VALUES  
(10, 'toto', 'CLERCK', null, null, 0, 0, 10)  
ERROR at line 1:  
check constraint (COURS1.CHK_EMP1_SAL) violated
```

Création de contraintes :

Contraintes de clé unique

- Les contraintes de clé unique (UNIQUE) :
 - permettent d'assurer l'unicité d'une colonne ou d'un groupe de colonnes,
 - les valeurs NULL sont autorisées,
 - les index uniques sont créés automatiquement à partir de la clé unique,
 - une clé unique ne peut pas être référencée par une clé étrangère.

Création de contraintes :

Contraintes de clé unique (suite)

- Exemple :

```
CREATE TABLE emp1
(
    empno          NUMERIC(4)  NOT NULL,
    ename          VARCHAR(10),
    job            VARCHAR(9),
    mgr            NUMERIC(4),
    Hiredate       DATE,
    sal            NUMERIC(7,2),
    comm           NUMERIC(7,2),
    deptno         NUMERIC(2)  NOT NULL,
    CONSTRAINT uq_emp1_ename UNIQUE(ename)
);
```


Création de contraintes :

Contraintes de clé primaire

- Les contraintes de clé primaire (PRIMARY KEY) :
 - toute table doit disposer d'une clé primaire (unique, mais pouvant être composée de plusieurs champs),
 - la norme SQL impose que toutes les colonnes d'une clé primaire soient obligatoires (la ou les colonne(s) est/sont forcée(s) à NOT NULL),
 - unicité des valeurs de la clé
 - clé généralement référencée par des clés étrangères.

Création de contraintes :

Contraintes de clé primaire (suite)

- Exemple :

```
CREATE TABLE emp1
(
    empno          NUMERIC(4) NOT NULL,
    ename          VARCHAR(10),
    job            VARCHAR(9),
    mgr            NUMERIC(4),
    hiredate       DATE,
    sal            NUMERIC(7,2),
    comm           NUMERIC(7,2),
    deptno         NUMERIC(2) NOT NULL,
    CONSTRAINT pk_emp1 PRIMARY KEY (empno)
)
```

Création de contraintes :

Contraintes de clé étrangère

- **Les contraintes de clé étrangère (FOREIGN KEY):**
 - une colonne ou un groupe de colonnes qui référencent la clé primaire d'une autre table,
 - la valeur de la clé étrangère doit :
 - exister dans la table référencée (valeur de la clé étrangère = une des valeurs de la clé primaire),
 - ou bien être NULL (sauf si elle est clé étrangère et clé primaire à la fois !).
 - une clé étrangère ne peut référencer une table d'une base distante.
 - **Le type de la clé étrangère doit correspondre à celui de la clé primaire**

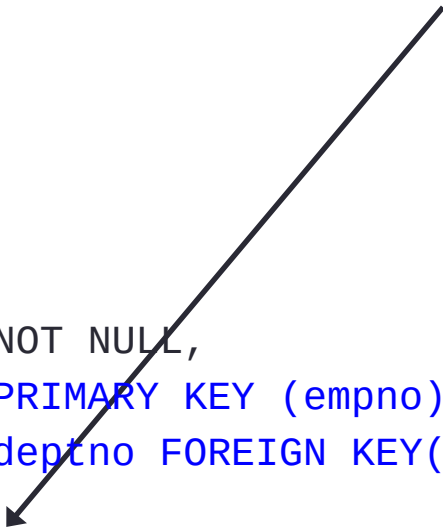
Création de contraintes :

Contraintes de clé étrangère (suite)

- Exemple :

```
CREATE TABLE emp1
(
  empno      NUMERIC(4)      NOT NULL,
  ename      VARCHAR(10),
  job        VARCHAR(9),
  mgr        NUMERIC(4),
  hiredate   DATE,
  sal        NUMERIC(7,2),
  comm       NUMERIC(7,2),
  deptno     NUMERIC(2)      NOT NULL,
  CONSTRAINT pk_emp1 PRIMARY KEY (empno),
  CONSTRAINT fk_emp1_deptno FOREIGN KEY(deptno)
    REFERENCES dept(deptno),
  CONSTRAINT fk_emp1_mgr FOREIGN KEY(mgr) REFERENCES
    emp1(empno)
);
```

Implique que DEPTNO soit clé primaire dans DEPT et que la table DEPT soit déjà créée !



Contrainte de clé étrangère : ON DELETE CASCADE, ON UPDATE CASCADE

- Exemple :

```
CREATE TABLE emp1
(
    empno          NUMERIC(4)          NOT NULL,
    ename          VARCHAR(10),
    job            VARCHAR(9),
    mgr            NUMERIC(4),
    hiredate       DATE,
    sal            NUMERIC(7,2),
    comm           NUMERIC(7,2),
    deptno         NUMERIC(2) NOT NULL,
    CONSTRAINT pk_emp1 PRIMARY KEY (empno),
    CONSTRAINT fk_emp1_deptno FOREIGN KEY(deptno)
        REFERENCES dept(deptno) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_emp1_mgr FOREIGN KEY(mgr) REFERENCES
        emp1(empno)
);
```

=> Si on supprime un service, de la table DEPT, référencé dans la table EMP, les lignes de table EMP référençant ce service sont aussi supprimées.

NULL et clé étrangère

- La contrainte de clé unique ne prend pas en compte les valeurs NULL (NULL n'est pas considérée comme une valeur !)
- Clés étrangères : la contrainte FOREIGN KEY n'est pas contrôlée pour une clé étrangère comportant la valeur NULL, y compris dans le cas d'une clé concaténée dont l'une des valeurs est NULL.

Clé étrangère
(Col1,Col2)

Col 1	Col 2	Col 3
X1	Y1	
X2	Y2	
X3	NULL	
NULL	NULL	
NULL	Y3	
X3	Y3	

Invalide

Clé primaire (Col1,Col2)

Col 1	Col 2
X1	Y1
X2	Y2

Remarque sur les contraintes

- Les contraintes dynamiques et les contraintes ensemblistes ne peuvent s'exprimer en SQL et doivent être prises en compte soit par le biais de triggers, soit dans le code des programmes d'applications (i.e. programmes clients).
- Exemple de contrainte ensembliste : le nombre d'employés par département (service) ne peut être supérieur à 10.

```
CREATE OR REPLACE FUNCTION ps_test_nb_employes() RETURNS TRIGGER AS
$BODY$
DECLARE
    vNbEmployes numeric;
BEGIN
    SELECT COUNT(*) INTO vNbEmployes FROM emp WHERE deptno=NEW.deptno;
    IF vNbEmployes = 10 THEN
        RAISE EXCEPTION '10 employés sont déjà présents dans ce
service';
    ELSE
        RETURN NEW;
    END IF;
END;
$BODY$
LANGUAGE 'plpgsql';
```

```
CREATE TRIGGER bf_insert_employe BEFORE INSERT ON emp
FOR EACH ROW EXECUTE PROCEDURE ps_test_nb_employes();
```

Modification de table : ALTER TABLE

- ALTER TABLE : permet de modifier la structure initiale d'une table
 - Ajout de colonnes,
 - Modification de la valeur par défaut,
 - Ajout de contraintes,
 - Activation, ou suppression de contraintes,
- Exemple : Changement de la définition de la colonne DNAME de DEPT


```
ALTER TABLE dept ALTER dname TYPE VARCHAR(20);
ALTER TABLE dept ALTER dname SET NOT NULL;
ALTER TABLE dept ALTER dname DROP NOT NULL;
-- Commande ALTER non normalisée (dépend du SGBD). Ex.
avec MySQL :
ALTER TABLE dept CHANGE dname
dname VARCHAR(20) NOT NULL;
```
- Exemple : ajout d'une colonne à la table DEPT


```
ALTER TABLE dept
ADD COLUMN date_creation DATE NULL
```


Modification de table : ALTER TABLE (suite)

- Exemple : ajout d'une contrainte de clé primaire à la table DEPT

```
ALTER TABLE dept  
ADD CONSTRAINT pk_dept PRIMARY KEY (deptno);
```

- Exemple : ajout d'une contrainte de clé étrangère à la table EMP

```
ALTER TABLE emp  
ADD CONSTRAINT fk_emp_deptno FOREIGN KEY (deptno)  
REFERENCES dept(deptno);
```

- Exemple : ajout d'une contrainte CHECK salaire > commission

```
ALTER TABLE emp  
ADD CONSTRAINT ck_emp_sal CHECK (SAL > COMM);
```

Modification de table : ALTER TABLE (suite)

- **Remarque :**

- Il est préférable de créer les contraintes de clé étrangère avec la commande ALTER TABLE, i.e. après la création des tables, plutôt que d'utiliser les contraintes déclaratives dans les CREATE TABLE. Si les clés étrangères sont créées en fin de script, il ne sera pas nécessaire de respecter l'ordre de création des tables.

Suppression de table : DROP TABLE

- Exemple : suppression de la table DEPT

`DROP TABLE dept;`

NOTICE: constraint fk_emp_deptno on table emp depends on table dept

ERROR: cannot drop table dept because other objects depend on it

HINT: Use DROP ... CASCADE to drop the dependent objects too.

=> Nécessité de supprimer les tables dans l'ordre adéquat (inverse de leur création) afin de respecter les contraintes clé primaire / clé étrangère (de même pour les enregistrements !)

Ou alors utiliser l'option `CASCADE` qui supprime les clés étrangères dépendantes : `DROP TABLE dept CASCADE;`

- Impossibilité de supprimer une contrainte avec un DROP.

Il faut utiliser `ALTER TABLE ... DROP ...`

- Exemple : `ALTER TABLE emp DROP CONSTRAINT fk_emp_deptno;`

SÉQUENCE

Séquence

- = générateur de nombre
- ⇔ AUTO_INCREMENT dans MySQL
- Vous devez utiliser une séquence à chaque fois que vous voulez qu'un numéro s'incrémente automatiquement
- Des tables différentes peuvent utiliser la même séquence. Sinon, possibilité de créer une séquence pour chaque table ayant un numéro automatique
- Après création d'une séquence, les fonctions `nextval`, `currval` et `setval` sont utilisées pour agir sur la séquence.

- Syntaxe (partielle) :

```
CREATE SEQUENCE nom [ INCREMENT [ BY ] incrément ]
[ MINVALUE valeurmin | NO MINVALUE ]
[ MAXVALUE valeurmax | NO MAXVALUE ]
[ START [ WITH ] début ]
[ OWNED BY { table.colonne | NONE } ]
```

Séquence (suite)

- `INCREMENT BY incrément` : précise la valeur à ajouter à la valeur courante de la séquence pour créer une nouvelle valeur. Valeur >0 (seq. croissante) ou <0 (seq. décroissante). 1 est la valeur par défaut.
- `MINVALUE valeurmin` : détermine la valeur minimale de la séquence. Si cette clause n'est pas fournie ou si `NO MINVALUE`, valeur par défaut = 1 (séquence croissante) ou $-2^{63}-1$ (séq. décroissante)
- `MAXVALUE valeurmax` : détermine la valeur maximale de la séquence. Si cette clause n'est pas fournie ou si `NO MAXVALUE` est spécifié, valeur par défaut = $2^{63}-1$ ou -1 pour les séquences ascendantes et descendantes.
- `START WITH début` : permet à la séquence de démarrer n'importe où. La valeur de début par défaut est `valeurmin` pour les séquences ascendantes et `valeurmax` pour les séquences descendantes.
- `OWNED BY table.colonne`, `OWNED BY NONE` : permet d'associer la séquence à une colonne de table spécifique. De cette façon, la séquence sera automatiquement supprimée si la colonne (ou la table entière) est supprimée. `OWNED BY NONE`, valeur par défaut, indique qu'il n'y a pas d'association.

Séquence (suite)

- Exemples :

- Créer une séquence ascendante appelée `serie`, démarrant à 101 :

```
CREATE SEQUENCE serie START WITH 101;
```

- Récupérer le prochain numéro de cette séquence :

```
SELECT nextval('serie');
```

```
nextval
```

```
-----
```

```
101
```

- Récupérer le prochain numéro d'une séquence :

```
SELECT nextval('serie');
```

```
nextval
```

```
-----
```

```
102
```

- Utiliser cette séquence dans une commande INSERT :

```
INSERT INTO matable (num, val) VALUES (nextval('serie'),  
    'toto');
```

```
-- Insère 103, 'toto' dans MATABLE
```

```
INSERT INTO matable2 (numero, valeur) VALUES (nextval('serie'),  
    'titi'); -- Insère 104, 'titi' dans MATABLE2
```

Séquence (suite)

- Exemples :

- Créer une séquence ascendante appelée seq_dept (valeur max = 100), associée à la table DEPT :

```
CREATE SEQUENCE seq_dept MAXVALUE 100 OWNED BY  
DEPT.deptno;
```

- Insérer un nouveau département :

```
INSERT INTO DEPT(deptno, dname, loc) VALUES  
(nextval('seq_dept'), 'Sce marketing', 'Annecy');
```

```
-- Insère 1, 'Sce marketing', 'Annecy'
```

- Suppression de la table DEPT :

```
DROP TABLE DEPT;
```

```
-- Supprime également la séquence car option OWNED BY utilisée, sinon :
```

```
DROP SEQUENCE seq_dept;
```


Séquence (suite)

- Il est possible de remplacer une séquence en utilisant un type sérié SERIAL ou BIGSERIAL :
 - SERIAL (entier à incrémentation automatique variant de 1 à 2147483647). Utiliser un type INTEGER dans la clé étrangère.
 - BIGSERIAL (entier de grande taille à incrémentation automatique variant de 1 à 9223372036854775807). Utiliser un type BIGINT dans la clé étrangère.
 - On peut aussi remplacer SERIAL et BIGSERIAL par la syntaxe normalisée (ANSI) :
 - SERIAL par INTEGER **GENERATED BY DEFAULT AS IDENTITY**
 - BIGSERIAL par BIGINT **GENERATED BY DEFAULT AS IDENTITY**

- Exemple :

```
CREATE SEQUENCE seq_dept; -- Créé la séquence seq_dept
CREATE TABLE DEPT(
    DEPTNO NUMERIC(2) NOT NULL DEFAULT nextval('seq_dept'),
    DNAME VARCHAR(14),
    LOC VARCHAR(13) );
ALTER SEQUENCE seq_dept OWNED BY DEPT.deptno; -- Modifie la
séquence pour l'associer au champ deptno de DEPT
```



```
CREATE TABLE DEPT(
    DEPTNO SERIAL NOT NULL,
    DNAME VARCHAR(14),
    LOC VARCHAR(13) );
```

-- NOTICE: CREATE TABLE will create implicit sequence "dept_deptno_seq" for serial column "dept.deptno"

Séquence (suite)

- Insertion dans la table DEPT :

```
INSERT INTO DEPT(dname, loc) VALUES ('Sce marketing',  
    'Annecy');
```

```
-- Insère 1, 'Sce marketing', 'Annecy'
```

```
INSERT INTO DEPT(dname, loc) VALUES ('Sce financier', 'Paris');
```

```
-- Insère 2, 'Sce financier', 'Paris'
```

VUE

Création de vue

- Syntaxe :

```
CREATE VIEW <nom_de_la_vue>  
AS <commande_SELECT>
```
- Les vues permettent :
 - une indépendance fonctionnelle et logique
 - modification des tables sans modification de l'application qui utilise la vue et non directement la table, rapprochement du langage utilisateur
 - la simplification d'utilisation pour l'utilisateur
 - jointures, CASE, SUM... déjà faits dans la création de la vue
 - la confidentialité
 - restriction du nombre de colonnes, du nombre de lignes suivant l'utilisateur.
- Une vue
 - n'occupe pas de place : seul l'ordre SELECT est stocké dans le dictionnaire,
 - n'est pas une autre table,
 - ne possède pas de données : les données restent stockées dans les tables faisant partie de l'ordre SELECT

Création de vue (suite)

- Exemple : numéro et nom des employés du service 10

```
CREATE VIEW v_dept10 AS
  SELECT      empno, ename
  FROM        emp
  WHERE       deptno = 10
```

- Dans l'exemple précédent, le nom des colonnes sera celui des colonnes dans le SELECT

```
SELECT      *
FROM        v_dept10
```

```
EMPNO  ENAME
```

```
-----
```

```
7782  CLARK
```

```
7839  KING
```

```
7934  MILLER
```

Création de vue (suite)

- Exemple : création d'une vue donnant la moyenne des salaires, les salaires mini et maxi et le nombre de salaires par service.

```
CREATE VIEW v_dept_ana (nom_dept, moyenne_sal, min_sal,
    max_sal, nb_sal) AS
```

```
SELECT    d.dname,
          AVG(e.sal),
          MIN(e.sal),
          MAX(e.sal),
          COUNT(e.sal)
```

```
FROM      dept d
  JOIN emp e ON e.deptno = d.deptno
```

```
GROUP BY dname
```

-- Dans cet exemple, le nom des colonnes est obligatoire du fait de l'utilisation de fonctions (AVG, MIN, ..)

- Résultat :

```
SELECT * FROM v_dept_ana
```

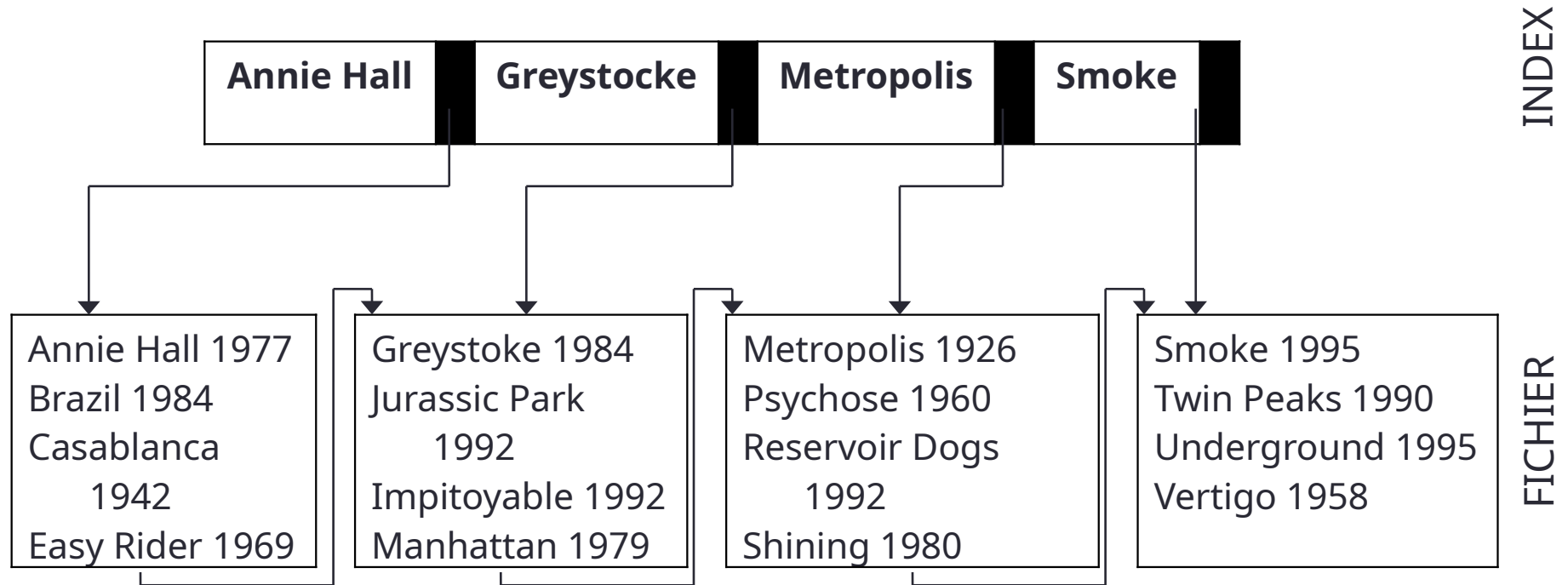
NOM_DEPT	MOYENNE_SAL	MIN_SAL	MAX_SAL	NB_SAL
ACCOUNTING	2916.66667	1300	5000	3
RESEARCH	2175	800	3000	5

INDEX

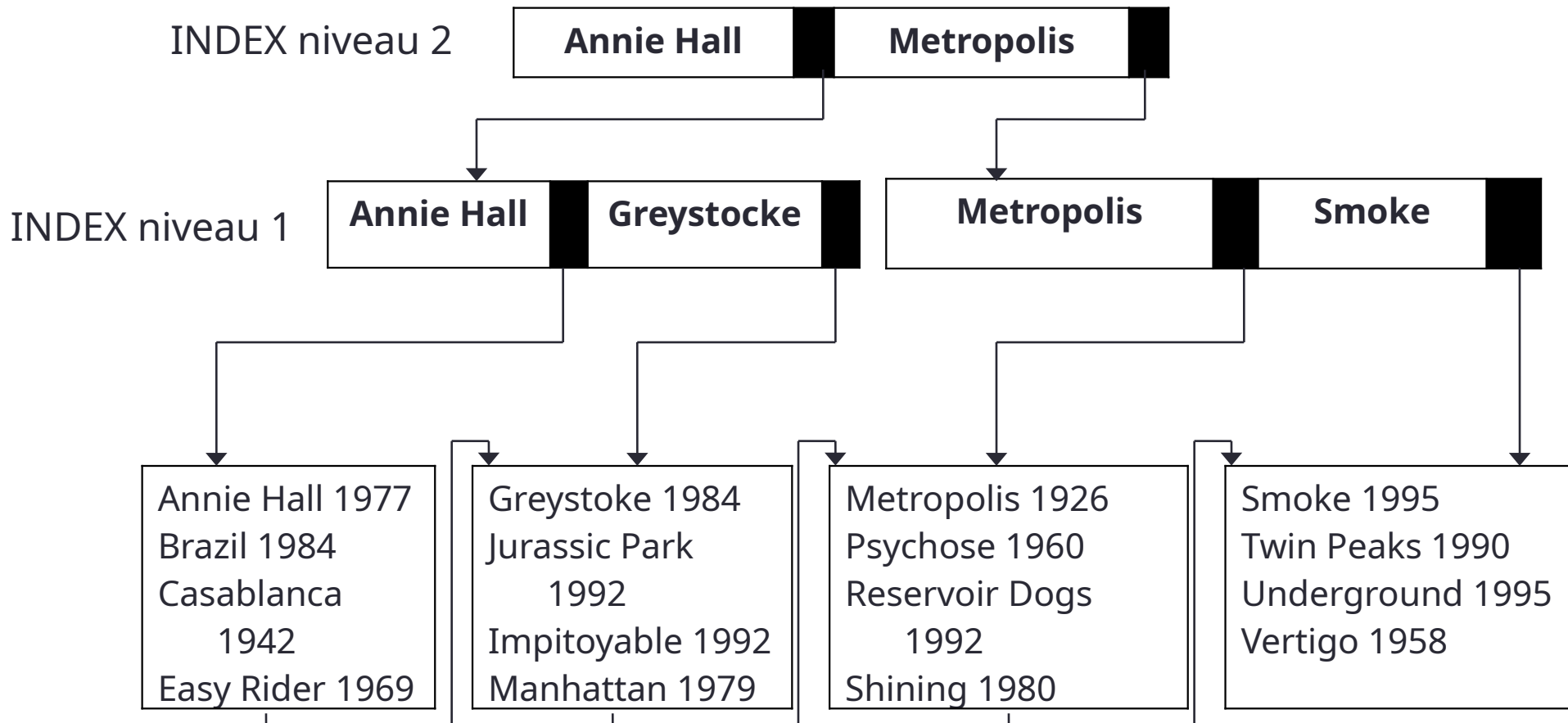
Index

- Les index permettent :
 - d'améliorer les performances en réduisant les lectures de blocs nécessaires pour retrouver les données,
 - de garantir l'unicité (INDEX UNIQUE).
- **Index** toujours positionné sur un fichier trié. Le(s) champ(s) sur le(s)quel(s) le fichier est trié est appelé **clé** ($\neq$ clé primaire).
- Un index est un *second fichier* possédant les caractéristiques suivantes:
 - Les articles sont des couples (*valeurdecléderecherche*, *adressedepage*)
 - Une occurrence (v , b) (ou *entrée*) dans un index signifie que le premier enregistrement dans la page **b** du fichier trié a pour valeur de clé **v** .
- L'index est lui-même **trié** sur la valeur de clé.

Exemple fichier indexé



Index multi-niveaux : cas des index volumineux



Création d'index

- **CREATE INDEX** : pour créer un index, il faut définir
 - Le nom de l'index,
 - Le nom de la table sur lequel porte l'index,
 - Le nom des colonnes de la table (une ou plusieurs colonnes),
 - Le type de l'index :
 - UNIQUE
 - NON UNIQUE
- **Exemple** : création d'un index sur la table EMP, colonne DEPTNO (foreign key)

```
CREATE INDEX idx_emp_deptno ON emp(deptno)
```
- **Exemple** : Création d'un index unique portant sur deux colonnes

```
CREATE UNIQUE INDEX idx_dept_deptno_dname  
ON dept(deptno, dname)
```

Quand créer des index ?

- PK : par défaut, les SGBD placent des index uniques sur les PK.
- FK : Les SGBD n'indexent pas les FK => les indexer (remarque : par défaut, PowerAMC crée des index sur les FK lors de la génération du script de création des tables). Index non uniques.
- Clés alternatives : index à créer manuellement pour les attributs fréquemment utilisés dans les clauses WHERE des instructions SELECT et UPDATE si les valeurs sont discriminantes (nombreuses valeurs distinctes)
- Eviter les valeurs NULL sur les attributs indexés
- Remarques :
 - L'index n'est utilisé que si la ou les colonnes font partie de la clause WHERE ou dans le cas des JOINTURES
 - L'utilisation de fonctions sur ces colonnes dans la clause WHERE inhibe l'utilisation de l'index.
 - Les index concaténés (portant sur plusieurs colonnes) sont recommandés lorsque celles-ci figurent souvent ensemble dans la clause WHERE **à condition que le SGBD sache les gérer...**